PRACTICAL NO 4

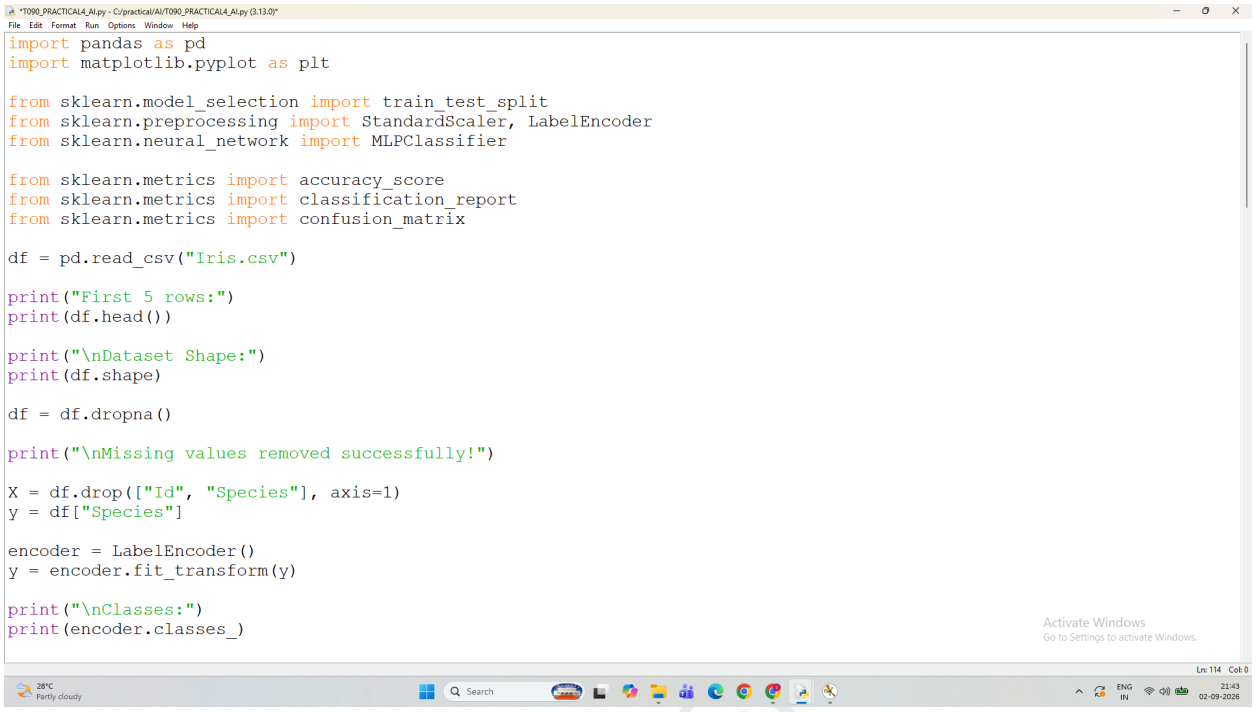
Aim :- Feed Forward Backpropagation Neural Network

1. Implement the Feed Forward Backpropagation algorithm to train a neural network.
2. Use the Iris dataset to train the neural network for a specific classification task.
3. Evaluate the performance of the trained neural network on test data.

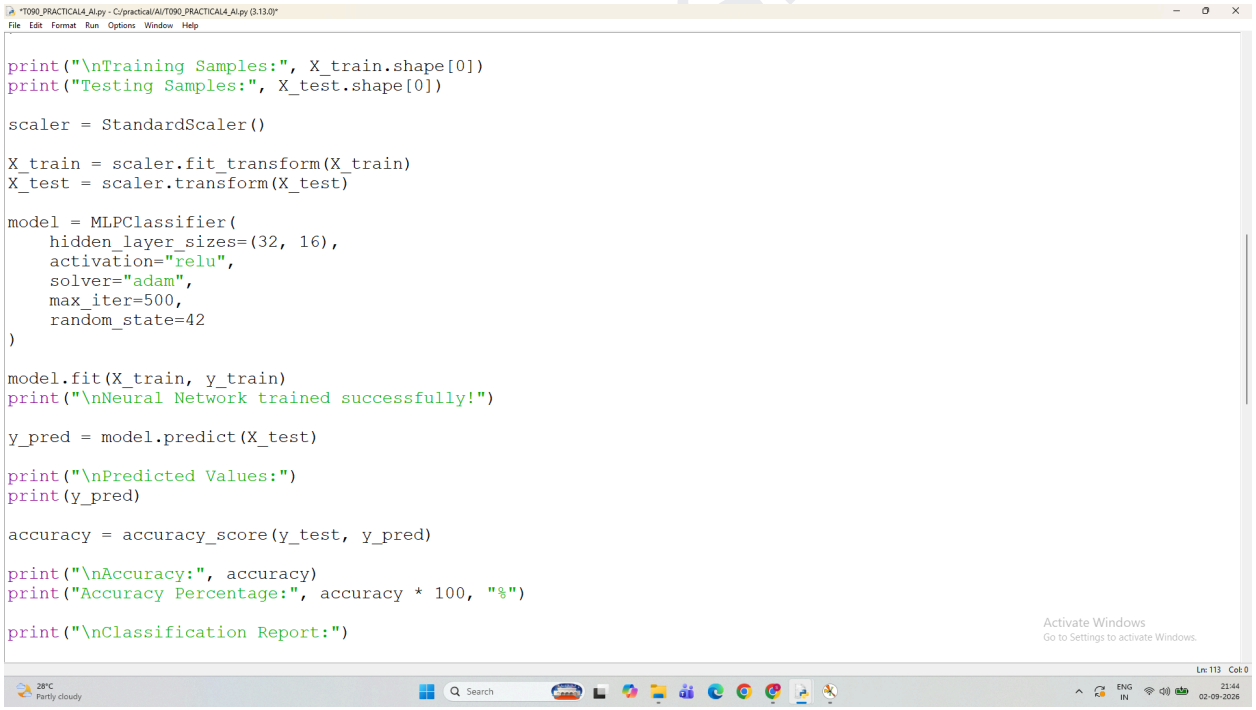
Step	Code / Function	Description
1	`pd.read_csv()`	Loads the Iris dataset.
2	`df.dropna()`	Removes rows containing missing values.
3	`X and y`	Separates input features and target variable.
4	`LabelEncoder()`	Converts target classes into numerical labels.
5	`train_test_split()`	Divides data into 80% training and 20% testing data.
6	`StandardScaler()`	Scales the input features for better neural-network training.
7	`MLPClassifier()`	Creates the Feed Forward Neural Network with hidden layers.
8	`model.fit()`	Trains the neural network using backpropagation.

9	`model.predict()`	Predicts Iris species on test data.
10	`accuracy_score()`	Calculates the accuracy of the trained model.
11	`classification_report()`	Displays precision, recall, and F1-score.
12	`confusion_matrix()`	Shows correct and incorrect predictions.
13	`plt.imshow()`	Visualizes the confusion matrix.
14	`model.loss_curve_`	Shows the training loss during neural-network learning.
15	`plt.plot()`	Visualizes the training loss curve.
16	`plt.colorbar()`	Displays the scale of values in the confusion matrix.
17	`plt.show()`	Displays the generated graphs.

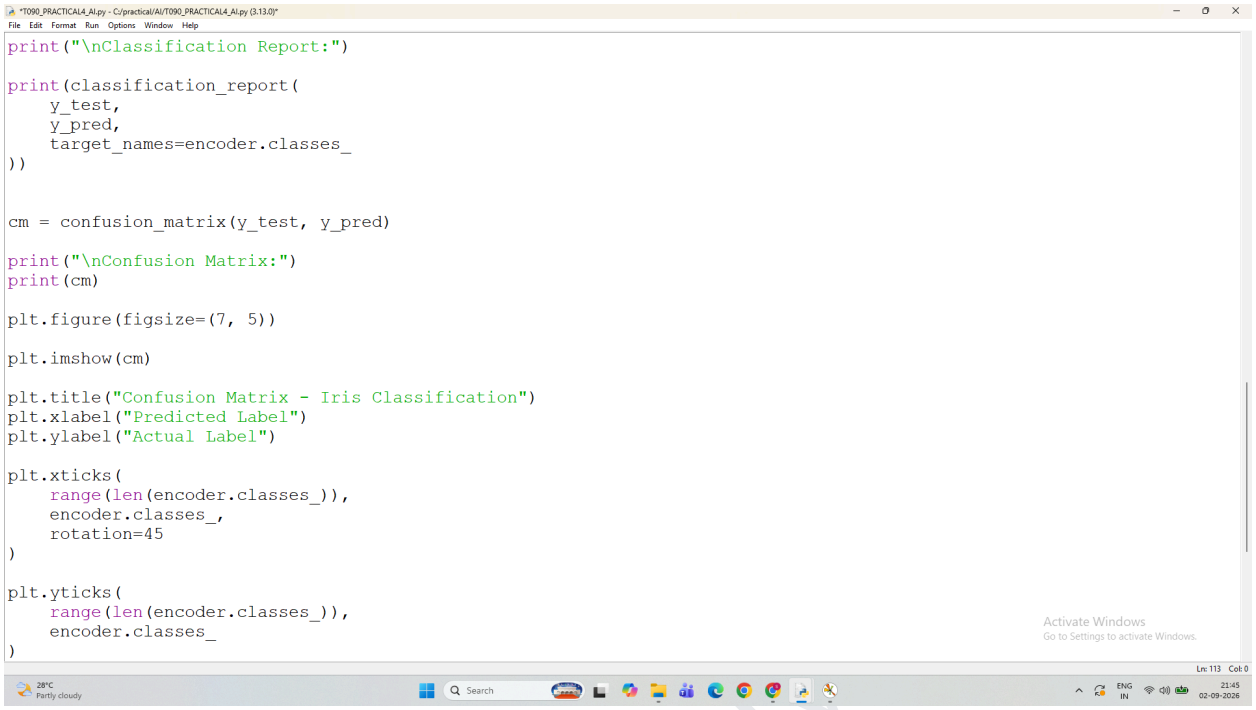
CODE:



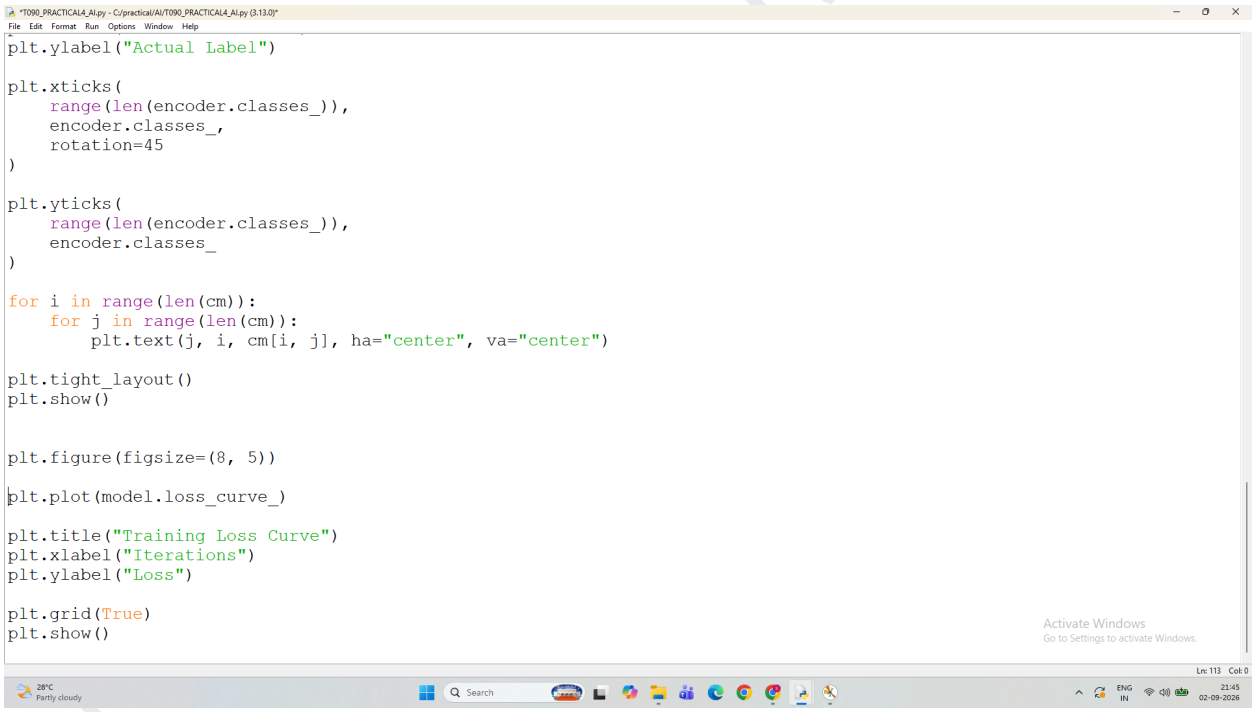
```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.neural_network import MLPClassifier

from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix

df = pd.read_csv("Iris.csv")

print("First 5 rows:")
print(df.head())

print("\nDataset Shape:")
print(df.shape)

df = df.dropna()

print("\nMissing values removed successfully!")

X = df.drop(["Id", "Species"], axis=1)
y = df["Species"]

encoder = LabelEncoder()
y = encoder.fit_transform(y)

print("\nClasses:")
print(encoder.classes_)
```



```
print("\nTraining Samples:", X_train.shape[0])
print("Testing Samples:", X_test.shape[0])

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = MLPClassifier(
    hidden_layer_sizes=(32, 16),
    activation="relu",
    solver="adam",
    max_iter=500,
    random_state=42
)

model.fit(X_train, y_train)
print("\nNeural Network trained successfully!")

y_pred = model.predict(X_test)

print("\nPredicted Values:")
print(y_pred)

accuracy = accuracy_score(y_test, y_pred)

print("\nAccuracy:", accuracy)
print("Accuracy Percentage:", accuracy * 100, "%")

print("\nClassification Report:")
```

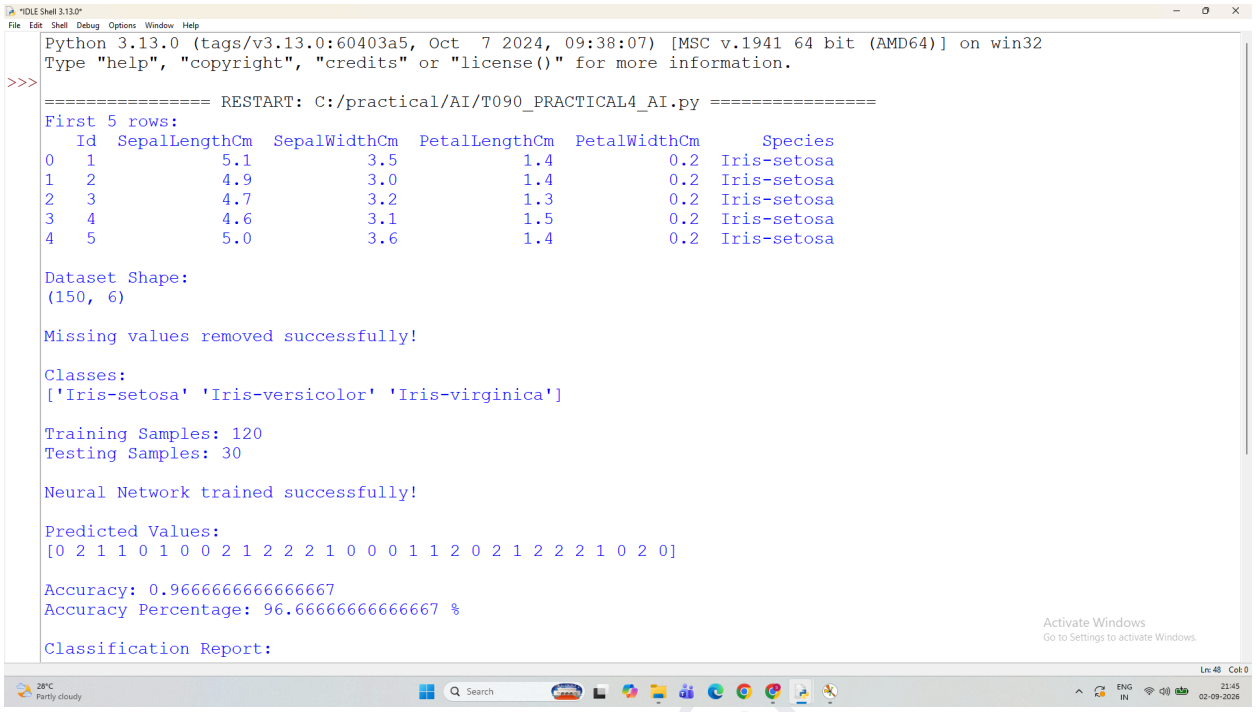


```
print("\nClassification Report:")

print(classification_report(
    y_test,
    y_pred,
    target_names=encoder.classes_
))


cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

plt.figure(figsize=(7, 5))

plt.imshow(cm)

plt.title("Confusion Matrix - Iris Classification")
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")

plt.xticks(
    range(len(encoder.classes_)),
    encoder.classes_,
    rotation=45
)

plt.yticks(
    range(len(encoder.classes_)),
    encoder.classes_
)
```

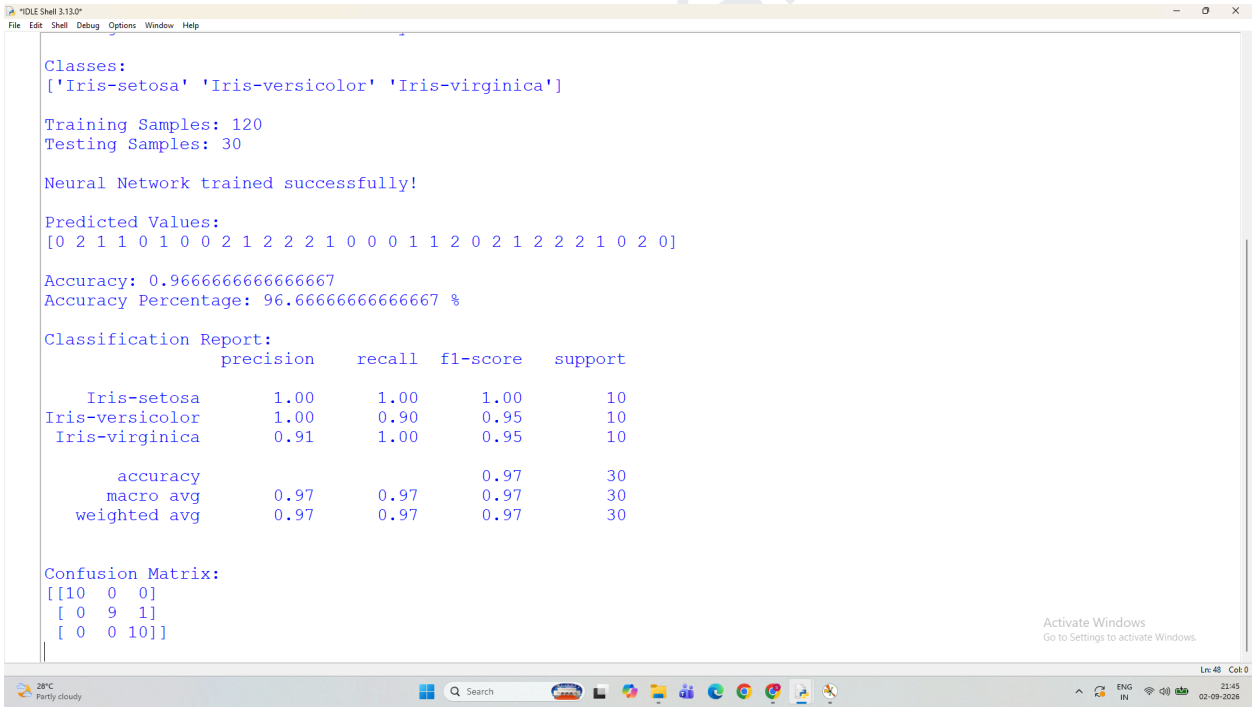


```
plt.ylabel("Actual Label")

plt.xticks(
    range(len(encoder.classes_)),
    encoder.classes_,
    rotation=45
)

plt.yticks(
    range(len(encoder.classes_)),
    encoder.classes_
)

for i in range(len(cm)):
    for j in range(len(cm)):
        plt.text(j, i, cm[i, j], ha="center", va="center")

plt.tight_layout()
plt.show()


plt.figure(figsize=(8, 5))

plt.plot(model.loss_curve_)

plt.title("Training Loss Curve")
plt.xlabel("Iterations")
plt.ylabel("Loss")

plt.grid(True)
plt.show()
```

OUTPUT:



```
Python 3.13.0 (tags/v3.13.0:60403a5, Oct  7 2024, 09:38:07) [MSC v.1941 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
================ RESTART: C:/practical/AI/T090_PRACTICAL4_AI.py ================
First 5 rows:
   Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm      Species
0   1            5.1           3.5            1.4           0.2  Iris-setosa
1   2            4.9           3.0            1.4           0.2  Iris-setosa
2   3            4.7           3.2            1.3           0.2  Iris-setosa
3   4            4.6           3.1            1.5           0.2  Iris-setosa
4   5            5.0           3.6            1.4           0.2  Iris-setosa

Dataset Shape:
(150, 6)

Missing values removed successfully!

Classes:
['Iris-setosa' 'Iris-versicolor' 'Iris-virginica']

Training Samples: 120
Testing Samples: 30

Neural Network trained successfully!

Predicted Values:
[0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 2 0 2 1 2 2 2 1 0 2 0]

Accuracy: 0.9666666666666667
Accuracy Percentage: 96.66666666666667 %

Classification Report:
```



```
Classes:
['Iris-setosa' 'Iris-versicolor' 'Iris-virginica']

Training Samples: 120
Testing Samples: 30

Neural Network trained successfully!

Predicted Values:
[0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 2 0 2 1 2 2 2 1 0 2 0]

Accuracy: 0.9666666666666667
Accuracy Percentage: 96.66666666666667 %

Classification Report:
                precision    recall  f1-score   support

    Iris-setosa       1.00      1.00      1.00        10
Iris-versicolor       1.00      0.90      0.95        10
 Iris-virginica       0.91      1.00      0.95        10

       accuracy                           0.97        30
      macro avg       0.97      0.97      0.97        30
   weighted avg       0.97      0.97      0.97        30


Confusion Matrix:
[[10  0  0]
 [ 0  9  1]
 [ 0  0 10]]
```

GRAPH:

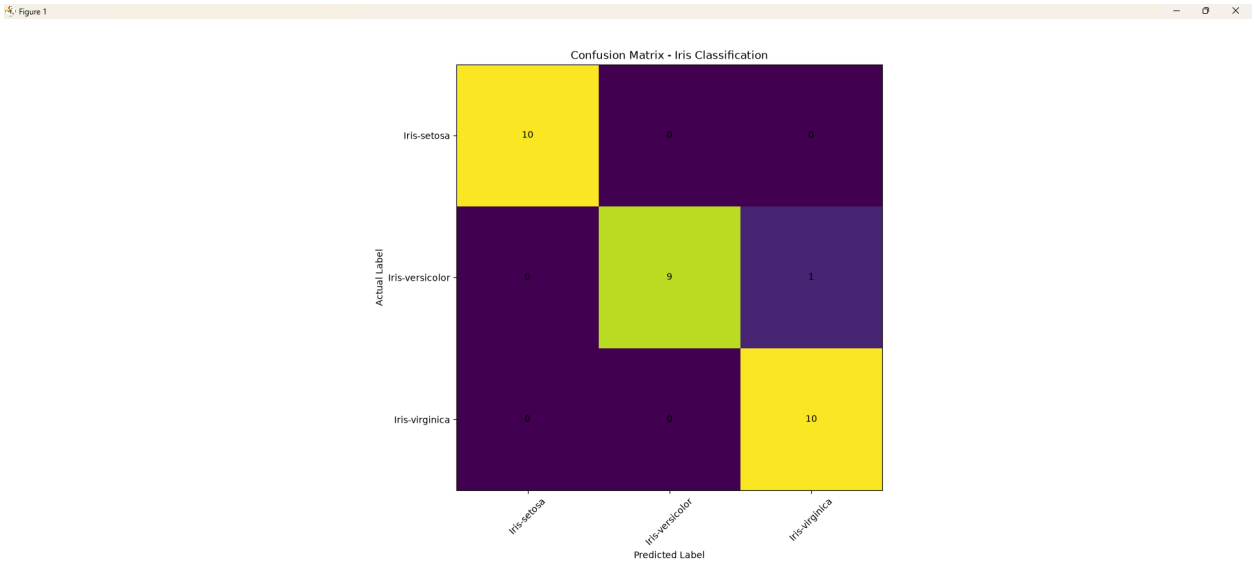


Figure 1
Confusion Matrix - Iris Classification
Iris-setosa
Iris-versicolor
Iris-virginica
Actual Label
10
0
0
0
9
1
0
0
10
Iris-setosa
Iris-versicolor
Iris-virginica
Predicted Label

Activate Windows
Go to Settings to activate Windows.
(x, y) = (,)
[0.00]
28°C
Partly cloudy
Search
ENG
IN
21:46
02-09-2026

Figure 1
Training Loss Curve
Loss
1.4
1.2
1.0
0.8
0.6
0.4
0.2
0.0
0
100
200
300
400
Iterations
Activate Windows
Go to Settings to activate Windows.
(x, y) = (165.8, 1.278)
28°C
Partly cloudy
Search
ENG
IN
21:46
02-09-2026

Observation :-

• **Confusion Matrix:** Most predictions are correctly classified, showing good model performance.

• **Training Loss Curve:** The loss decreases during training, indicating that the neural network learns effectively.

• **Overall Observation:** The Feed Forward Backpropagation Neural Network provides satisfactory classification of Iris species.